МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КАЗАХСТАН

«НАО Евразийский национальный университет имени Л. Н. Гумилёва»

Факультет информационных технологий

Кафедра Информационная безопасность

ОТЧЕТ

по практической работе № 5

Дисциплина: «Клиент-серверные технологии и их безопасность»

Тема: «CI/CD Security Pipeline: настройка автоматического сканирования уязвимостей исходного кода (SAST) и зависимостей (SCA) в GitHub Actions / GitLab CI»

Руководитель, преподаватель _____ Казиева Н.М.

«___» _____ 2026

Выполнил магистрант группы МСИБ-24

Нуртазинов Ислам Тулегенович

«___» _____ 2026

Астана, 2026

1. Цель работы

Практическая: построить конвейер CI/CD, который при каждом изменении кода автоматически выполняет статический анализ исходного кода (SAST), анализ состава программного обеспечения и уязвимостей зависимостей (SCA), поиск секретов и тесты, а затем по формальной политике решает, можно ли принимать изменение.

Теоретическая: изучить подход DevSecOps и принцип «shift-left», классы уязвимостей, которые находят SAST и SCA, форматы обмена результатами (SARIF, отчёты GitLab) и ограничения автоматических сканеров; принципы проектирования ролевой модели доступа RBAC (NIST RBAC, принцип наименьших привилегий) и методы её негативного тестирования.

2. Задание

Вариант — тема № 46 из перечня мини-проектов: «CI/CD Security Pipeline: Настройка автоматического сканирования уязвимостей исходного кода (SAST) и зависимостей (SCA) в GitHub Actions / GitLab CI».

Объектом проверки выбран REST-сервис с ролевой моделью доступа (RBAC) из методических указаний к практической работе № 5: таблицы users, roles, permissions, user_roles, role_permissions и эндпоинты /api/v1/public, /api/v1/moderator/audit, /api/v1/admin/users. Требуется:

- подготовить исходную версию сервиса с типичными ошибками безопасности в коде и устаревшими зависимостями;
- описать пайплайн безопасности для GitHub Actions и для GitLab CI: SAST (Bandit, Semgrep), SCA (pip-audit, Trivy), поиск секретов (Gitleaks / Secret Detection), тесты;
- написать собственные правила SAST для проверки RBAC и единый Security Gate с политикой блокировки;
- выполнить сканирование, проанализировать находки, устранить уязвимости и показать, что после исправления пайплайн проходит.

Дополнительно выполнены обязательные шаги методических указаний ПР5:

- шаг 1 — SQL-скрипт таблиц users, roles, permissions, user_roles, role_permissions с FOREIGN KEY и ON DELETE CASCADE, ER-диаграмма;
- шаг 2 — middleware проверки разрешений и три уровня ресурсов: /api/v1/public, /api/v1/moderator/audit, /api/v1/admin/users;
- шаг 3 — позитивные и негативные тесты в Postman (200/201, 403), а также проверка смены роли «на лету» без перевыпуска токена.

3. Теоретические сведения

3.1. DevSecOps и принцип «shift-left»

DevSecOps — практика встраивания проверок безопасности в каждый этап жизненного цикла разработки, а не в отдельный аудит перед выпуском. Принцип «shift-left» означает перенос проверок как можно ближе к моменту написания кода: уязвимость, найденная в pull request, исправляется за минуты, а найденная в эксплуатации — требует расследования инцидента, выпуска патча и уведомления пользователей. Пайплайн CI/CD — естественное место для таких проверок: он запускается при каждом изменении, одинаково для всех разработчиков, а его результат можно сделать обязательным условием слияния (branch protection).

3.2. SAST — статический анализ исходного кода

SAST (Static Application Security Testing) анализирует исходный код без его запуска: строит абстрактное синтаксическое дерево (AST), граф потока данных и ищет опасные конструкции — подстановку пользовательских данных в SQL-запрос или команду ОС, небезопасную десериализацию, слабые криптоалгоритмы, жёстко заданные секреты. Преимущества — полнота покрытия кода и точное указание файла и строки; недостатки — ложные срабатывания и слепота к ошибкам бизнес-логики, не описанным

правилами (например, к забытой проверке роли). Последнее компенсируется собственными правилами.

Bandit — анализатор Python от PyCQA: набор плагинов B1xx–B7xx по AST, у каждой находки есть уровень серьёзности (severity) и достоверности (confidence).

Semgrep — многоязычный анализатор с правилами в виде шаблонов кода (YAML). Поддерживает наборы правил сообщества (p/python, p/owasp-top-ten) и позволяет быстро писать собственные правила под архитектуру проекта.

3.3. SCA — анализ состава ПО и зависимостей

SCA (Software Composition Analysis) строит перечень сторонних компонентов приложения, включая транзитивные зависимости, и сопоставляет их версии с базами известных уязвимостей: CVE/NVD, GitHub Advisory Database (GHSA), Python Packaging Advisory Database (PYSEC) и агрегатором OSV. Для каждой уязвимости известна оценка CVSS и версия, в которой она исправлена. Это соответствует категории OWASP Top 10 A06:2021 «Vulnerable and Outdated Components». Особенность SCA — результат меняется без изменения кода: новая CVE в используемой библиотеке появляется в любой день, поэтому пайплайн запускается и по расписанию.

pip-audit (PyPA) разрешает полное дерево зависимостей requirements.txt и проверяет его по базе PyPI/OSV. **Trivy** (Aqua Security) — универсальный сканер файловой системы, образов контейнеров и IaC с собственной базой уязвимостей.

3.4. Поиск секретов и формат SARIF

Утечка ключей и паролей в репозиторий — отдельный класс проблем: секрет остаётся в истории git даже после удаления из текущей версии. Gitleaks и GitLab Secret Detection проверяют всю историю коммитов по сигнатурам и энтропии строк.

SARIF (Static Analysis Results Interchange Format, OASIS) — стандартный JSON-формат результатов статического анализа. GitHub принимает SARIF через действие upload-sarif и показывает находки во вкладке Security → Code scanning и прямо в diff pull request. GitLab использует собственные форматы отчётов (artifacts:reports:sast, dependency_scanning), которые отображаются в виджете Merge Request и в Security Dashboard.

Таблица 1 — Сравнение видов автоматического тестирования безопасности

Подход	**Что анализирует**	**Когда**	**Примеры находок**	**Инструменты**
SAST	исходный код (AST, потоки данных)	commit / PR	SQLi, OS command injection, слабый хеш, секрет в коде	Bandit, Semgrep, CodeQL
SCA	зависимости и их версии	commit / PR + расписание	CVE в библиотеках, лицензии	pip-audit, Trivy, Dependabot
Secrets	история git	commit / PR	API-ключи, пароли, токены	Gitleaks, Secret Detection
DAST	работающее приложение	после деплоя на стенд	ошибки конфигурации, XSS, IDOR	OWASP ZAP, Burp

3.5. Ролевая модель доступа RBAC

RBAC (Role-Based Access Control) — модель, в которой права (permissions) выдаются не пользователям напрямую, а ролям, а пользователь получает права через назначенные ему роли. Стандарт NIST RBAC (ANSI INCITS 359) выделяет четыре уровня модели.

Таблица 2 — Уровни модели NIST RBAC

Уровень NIST RBAC	**Содержание**	**Реализация в работе**
Core (базовый)	пользователи, роли, права, сеансы; связи многие-ко-многим	таблицы users, roles, permissions, user_roles, role_permissions
Hierarchical	иерархия ролей: старшая роль наследует права младшей	задана составом прав: ADMIN ⊃ MODERATOR ⊃ USER
Constrained (SSD/DSD)	разделение обязанностей: несовместимые роли	не требуется заданием; set_role назначает одну роль
Symmetric	обзор прав: какие роли и пользователи имеют право X	запрос GET /api/v1/admin/users и SQL-запрос прав пользователя

Принцип наименьших привилегий (PoLP) требует давать каждой роли только права, необходимые для её задач, и запрещать всё, что явно не разрешено (deny by default). Проверка выполняется на сервере для каждого

запроса: скрытие кнопок в интерфейсе защитой не является. Типичные ошибки — эндпоинт без проверки права (Broken Function Level Authorization, эскалация привилегий), доступ к чужому объекту по идентификатору (IDOR / BOLA) и возможность самому назначить себе роль через тело запроса (Mass Assignment).

4. Среда выполнения и инструменты

Таблица 3 — Среда выполнения и версии инструментов

Компонент	**Версия / значение**
ОС (локальный прогон)	Windows 11 Pro
Python (локально / в CI)	3.10.7 / 3.12
FastAPI · PyJWT · PyYAML (после исправления)	0.141.1 · 2.15.1 · 6.0.3
pytest · requests (клиент API-тестов)	9.2.0 · 2.34.2
Postman	коллекция и окружение в формате Postman Collection v2.1
СУБД	SQLite (демонстрационный стенд); SQL-скрипт для PostgreSQL
Bandit	1.9.4
Semgrep	1.183.0
pip-audit	2.10.1
Trivy (в CI)	aquasecurity/trivy-action v0.36.0, образ aquasec/trivy:0.67.2
Gitleaks (в CI / локально)	gitleaks/gitleaks-action@v3 / gitleaks 8.30.1
Репозиторий GitHub	github.com/is1am/cicd-security-pipeline
Система контроля версий	git, ветка main

Таблица 4 — Структура проекта ПР5_CICD_Security

Путь	**Назначение**
app/main.py, app/security.py, app/db.py	RBAC-сервис на FastAPI (объект проверки)
tests/test_rbac.py	функциональные и негативные тесты RBAC (pytest)
sql/rbac_schema.sql	SQL-скрипт схемы RBAC для PostgreSQL с начальными данными
postman/*.json	Postman-коллекция (21 запрос) и окружение
scripts/run_collection.py	запуск сервиса и выполнение Postman-коллекции
.github/workflows/security.yml	пайплайн GitHub Actions
.github/dependabot.yml	автоматические PR с обновлением зависимостей
.gitlab-ci.yml	пайплайн GitLab CI
.semgrep/rbac-rules.yml	собственные правила Semgrep
security-policy.json	политика Security Gate (пороги и исключения)
.gitleaks.toml	стандартные правила Gitleaks + исключение для reports/
scripts/security_gate.py	сводка отчётов и решение «пропустить / заблокировать»
scripts/run_local_scans.py	локальный прогон тех же проверок, что и в CI
reports/before, reports/after	отчёты сканеров до и после исправлений

5. Объект проверки: RBAC-сервис

5.1. Модель данных

Сервис реализует классическую модель RBAC из методических указаний: права (permissions) назначаются ролям, роли — пользователям. Связи многие-ко-многим реализованы таблицами user_roles и role_permissions с составными первичными ключами. Внешние ключи объявлены с ON DELETE CASCADE и ON UPDATE CASCADE: при удалении пользователя или роли связи удаляются автоматически, и «висячих» назначений прав не остаётся.

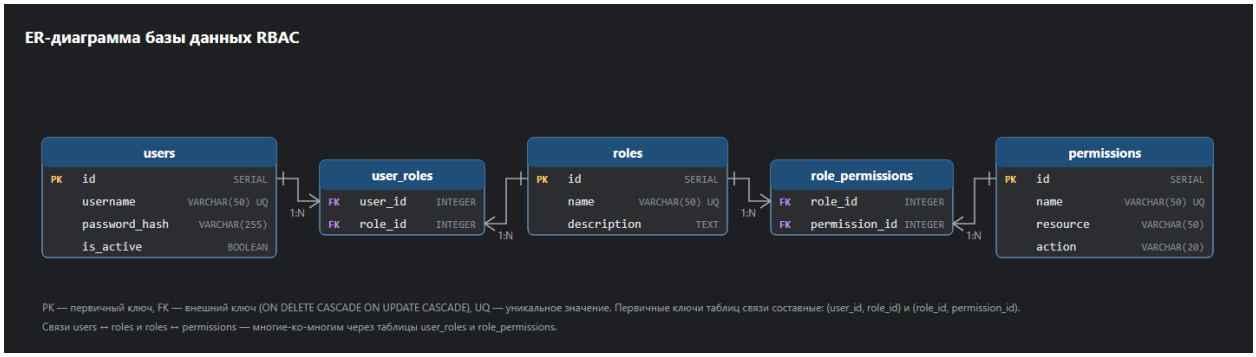


Рисунок 1 — ER-диаграмма базы данных RBAC

```
CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_active     BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE roles (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(50) NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE permissions (
    id       SERIAL PRIMARY KEY,
    name     VARCHAR(50) NOT NULL UNIQUE,
    resource VARCHAR(50) NOT NULL,
    action   VARCHAR(20) NOT NULL,
    UNIQUE (resource, action)
);

-- Связь многие-ко-многим: пользователь <-> роль
CREATE TABLE user_roles (
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE,
    role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE CASCADE ON UPDATE CASCADE,
    PRIMARY KEY (user_id, role_id)
);

-- Связь многие-ко-многим: роль <-> разрешение
CREATE TABLE role_permissions (
    role_id       INTEGER NOT NULL REFERENCES roles(id)       ON DELETE CASCADE ON UPDATE
CASCADE,
    permission_id INTEGER NOT NULL REFERENCES permissions(id) ON DELETE CASCADE ON UPDATE
CASCADE,
    PRIMARY KEY (role_id, permission_id)
);

CREATE INDEX idx_user_roles_role ON user_roles(role_id);
CREATE INDEX idx_role_permissions_perm ON role_permissions(permission_id);
```

Листинг 1 — SQL-скрипт создания таблиц (sql/rbac_schema.sql, PostgreSQL)

Таблица 5 — Матрица «роль — права»

Роль	**Права**
ADMIN	READ_PUBLIC, VIEW_AUDIT, EXPORT_AUDIT, MANAGE_USERS
MODERATOR	READ_PUBLIC, VIEW_AUDIT
USER	READ_PUBLIC

5.2. Middleware проверки прав

Проверка реализована зависимостями FastAPI. get_current_user проверяет подпись и срок действия JWT и то, что учётная запись существует и активна (is_active). Фабрика require_permission("ПРАВО") создаёт зависимость, которая при каждом запросе получает права пользователя из БД

одним SQL-запросом через user_roles и role_permissions и возвращает 403 Forbidden, если нужного права нет. Проверяется право, а не имя роли: добавление новой роли не требует правки кода эндпоинтов.

```
def get_current_user(authorization: str = Header(default="")):
    if not authorization.startswith("Bearer "):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Требуется токен")
    try:
        claims = decode_token(authorization[7:])
    except Exception:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Недействительный токен")
    with db.connect() as conn:
        row = conn.execute("SELECT is_active FROM users WHERE id = ?",
                           (int(claims["sub"]),)).fetchone()
    if not row or not row["is_active"]:  # удалённый или заблокированный пользователь
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Учётная запись неактивна")
    return {"id": int(claims["sub"]), "username": claims["name"]}



def require_permission(required: str):
    def dependency(user=Depends(get_current_user)):
        with db.connect() as conn:
            if required not in db.get_permissions(conn, user["id"]):
                raise HTTPException(status.HTTP_403_FORBIDDEN,
                                    "Недостаточно прав для выполнения операции")
        return user
    return dependency
```

Листинг 2 — проверка токена и разрешений (app/main.py)

```
@app.get("/api/v1/admin/users")
def list_users(user=Depends(require_permission("MANAGE_USERS"))):
    with db.connect() as conn:
        return db.list_users(conn)



@app.post("/api/v1/admin/users", status_code=201)
def create_user(body: UserIn, user=Depends(require_permission("MANAGE_USERS"))):
    try:
        with db.connect() as conn:
            uid = db.create_user(conn, body.username, body.password, body.role)
    except sqlite3.IntegrityError:
        raise HTTPException(status.HTTP_409_CONFLICT, "Пользователь уже существует")
    return {"id": uid, "username": body.username, "role": body.role}



@app.put("/api/v1/admin/users/{user_id}/role")
def change_role(user_id: int, body: RoleIn,
user=Depends(require_permission("MANAGE_USERS"))):
    with db.connect() as conn:
        if not db.set_role(conn, user_id, body.role):
            raise HTTPException(status.HTTP_404_NOT_FOUND, "Пользователь не найден")
    return {"id": user_id, "role": body.role}
```

Листинг 3 — административные эндпоинты, защищённые правом MANAGE_USERS

Роль в теле запроса POST /api/v1/admin/users ограничена типом Literal["USER", "MODERATOR", "ADMIN"]: неизвестное значение

отклоняется с кодом 422 до обращения к БД. Назначать роли может только обладатель MANAGE_USERS, поэтому модератор не может создать администратора (защита от Mass Assignment и эскалации).

5.3. Эндпоинты и намеренные уязвимости

Таблица 6 — Эндпоинты сервиса и права доступа

Эндпоинт	Требуемое право	Роли
POST /api/v1/auth/login	—	все (выдача JWT)
GET /api/v1/public	READ_PUBLIC	USER, MODERATOR, ADMIN
GET /api/v1/users/search	READ_PUBLIC	USER, MODERATOR, ADMIN
GET /api/v1/moderator/audit	VIEW_AUDIT	MODERATOR, ADMIN
GET /api/v1/moderator/audit/export	EXPORT_AUDIT	ADMIN
GET /api/v1/admin/users	MANAGE_USERS	ADMIN
POST /api/v1/admin/users	MANAGE_USERS	ADMIN
PUT /api/v1/admin/users/{id}/role	MANAGE_USERS	ADMIN
POST /api/v1/admin/import	MANAGE_USERS	ADMIN

В исходную версию (коммит a4be20f) намеренно внесены ошибки, которые часто встречаются в реальных проектах:

Таблица 7 — Уязвимости исходной версии

№	Ошибка	Где	CWE / OWASP
1	SQL-запрос собирается f-строкой из параметра q	main.py, /users/search	CWE-89 · A03 Injection
2	Вызов оболочки ОС с shell=True и параметром запроса	main.py, /audit/export	CWE-78 · A03 Injection
3	Эндпоинт экспорта не проверяет право EXPORT_AUDIT	main.py, /audit/export	CWE-285 · A01 Broken Access Control
4	yaml.load с yaml.Loader (создание произвольных объектов)	main.py, /admin/import	CWE-502 · A08
5	Секрет подписи JWT в исходном коде	security.py	CWE-798 · A07
6	JWT декодируется без проверки подписи	security.py	CWE-347 · A02
7	Пароли хешируются MD5 без соли	security.py	CWE-327/916 · A02
8	Сервер слушает 0.0.0.0	main.py	CWE-605 · A05
9	Устаревшие версии fastapi, PyJWT, requests, Jinja2	requirements.txt	A06 Vulnerable Components

6. Реализация пайплайна безопасности

6.1. Архитектура

Пайплайн запускается при push и pull request в ветку main, еженедельно по расписанию (чтобы обнаруживать новые CVE в неизменном коде) и

вручную. Проверки выполняются параллельно в отдельных джобах, каждая сохраняет машиночитаемый отчёт (JSON) и, где возможно, SARIF. Финальная джоба Security Gate собирает отчёты и по политике принимает решение. В настройках репозитория (branch protection) статус Security Gate делается обязательным, поэтому изменение с блокирующей находкой нельзя слить.

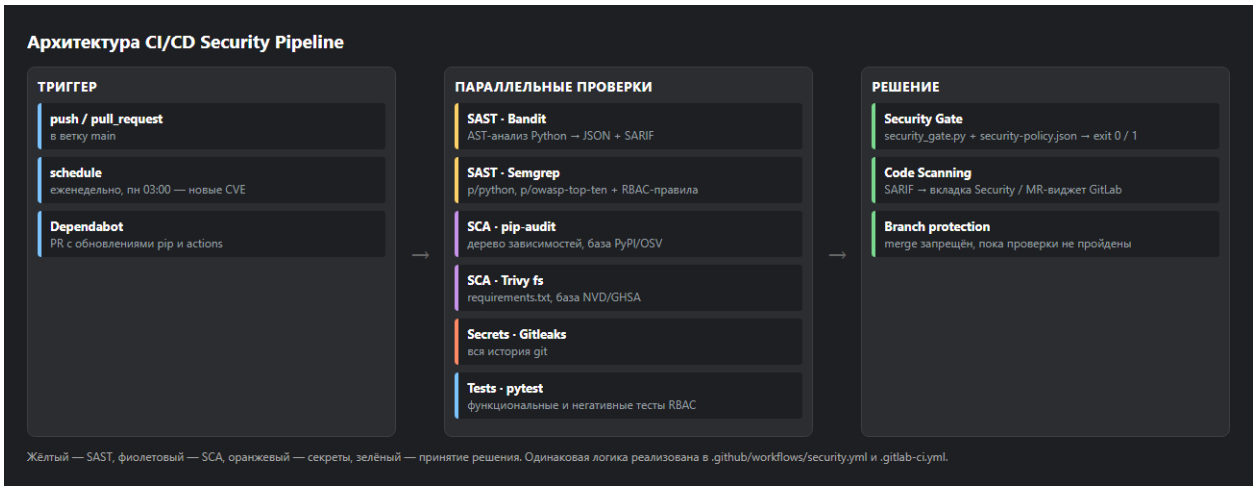


Рисунок 2 — Архитектура CI/CD Security Pipeline

6.2. GitHub Actions

Файл .github/workflows/security.yml. Основные решения:

- `permissions: contents: read` на уровне workflow и `security-events: write` только в джобах, загружающих SARIF — принцип наименьших привилегий для GITHUB_TOKEN;
- версии сканеров и Docker-образов зафиксированы (воспроизводимость, защита от подмены инструмента); для production рекомендуется закреплять actions по SHA коммита;
- Bandit, Semgrep и Trivy загружают SARIF в Code Scanning (category разделяет инструменты);
- Gitleaks получает полную историю (fetch-depth: 0);
- сканеры не прерывают джобу сами (--exit-zero / || true): решение принимает Security Gate по единой политике, а отчёты сохраняются как артефакты даже при наличии находок;
- `concurrency` отменяет устаревшие запуски той же ветки.

```
  sast-semgrep:
    name: SAST · Semgrep
    runs-on: ubuntu-latest
    permissions:
      contents: read
      security-events: write
    container:
      image: semgrep/semgrep:1.183.0
    steps:
      - uses: actions/checkout@v7
      - name: Semgrep (стандартные наборы + собственные RBAC-правила)
        run: >
          semgrep scan --metrics=off
          --config p/python --config p/owasp-top-ten
          --config .semgrep/rbac-rules.yml
          --json -o semgrep.json
          --sarif-output=semgrep.sarif
          app
      - uses: github/codeql-action/upload-sarif@v4
        if: always()
        with:
          sarif_file: semgrep.sarif
          category: semgrep
```

Листинг 4 — джоба Semgrep в GitHub Actions

```
  security-gate:
    name: Security Gate
    runs-on: ubuntu-latest
    needs: [sast-bandit, sast-semgrep, sca-pip-audit, sca-trivy]
    if: always()
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: ${{ env.PYTHON_VERSION }}
      - uses: actions/download-artifact@v8
        with:
          path: reports
          merge-multiple: true
      - name: Сводка и решение о пропуске сборки
        run: python scripts/security_gate.py reports --policy security-policy.json --
require-trivy --summary "$GITHUB_STEP_SUMMARY"
```

Листинг 5 — джоба Security Gate в GitHub Actions

6.3. GitLab CI

Файл .gitlab-ci.yml реализует ту же логику в стадиях test → security → gate. Встроенный шаблон Jobs/Secret-Detection подключается через include. Semgrep формирует отчёт в формате GitLab SAST (--gitlab-sast-output), Trivy — отчёт dependency_scanning по шаблону gitlab.tpl; оба публикуются через artifacts:reports и отображаются в Merge Request. Собственные джобы не требуют тарифа Ultimate.

```
sast:semgrep:
  stage: security
  image: $SEMGREP_IMAGE
  script:
    - >
      semgrep scan --metrics=off
      --config p/python --config p/owasp-top-ten
      --config .semgrep/rbac-rules.yml
      --json -o semgrep.json
      --gitlab-sast-output=gl-sast-report.json
      app
  artifacts:
    when: always
    paths: [semgrep.json]
    reports:
      sast: gl-sast-report.json
```

Листинг 6 — джоба Semgrep в GitLab CI

```
sca:trivy:
  stage: security
  image:
    name: $TRIVY_IMAGE
    entrypoint: [""]
  script:
    - trivy fs --scanners vuln --format json -o trivy.json .
    - trivy fs --scanners vuln --format template
        --template "@/contrib/gitlab.tpl" -o gl-dependency-scanning-report.json .
  artifacts:
    when: always
    paths: [trivy.json]
    reports:
      dependency_scanning: gl-dependency-scanning-report.json
```

Листинг 7 — джоба Trivy в GitLab CI

6.4. Собственные правила Semgrep для RBAC

Стандартные правила не знают, что в проекте права проверяются через require_permission. Поэтому написано правило rbac-privileged-route-without-permission: оно срабатывает на любой маршрут /api/v1/admin/* или /api/v1/moderator/*, у которого нет зависимости Depends(require_permission(...)) ни в параметрах функции, ни в dependencies декоратора. Так SAST начинает находить ошибку контроля доступа (OWASP API5 Broken Function Level Authorization), которую обычные сканеры пропускают. Ещё два правила запрещают декодирование JWT без проверки подписи и секрет в коде.

```
  - id: rbac-privileged-route-without-permission
    message: >-
      Привилегированный эндпоинт $PATH не проверяет разрешение через
      require_permission(...). Достаточно любого валидного токена —
      возможна эскалация привилегий (OWASP API5: Broken Function Level Authorization).
    severity: ERROR
    languages: [python]
    metadata:
      cwe: "CWE-285: Improper Authorization"
      owasp: "A01:2021 - Broken Access Control"
      category: security
    patterns:
      - pattern: |
          @$APP.$METHOD($PATH, ...)
          def $FUNC(...):
            ...
      - metavariable-regex:
          metavariable: $METHOD
          regex: ^(get|post|put|patch|delete)$
      - metavariable-regex:
          metavariable: $PATH
          regex: ^["']/api/v1/(admin|moderator)
      - pattern-not: |
          @$APP.$METHOD($PATH, ...)
          def $FUNC(..., $U = Depends(require_permission(...)), ...):
            ...
      - pattern-not: |
          @$APP.$METHOD($PATH, ..., dependencies=[..., Depends(require_permission(...)),
...], ...)
          def $FUNC(...):
            ...
```

Листинг 8 — правило Semgrep для проверки RBAC-маршрутов

6.5. Security Gate и политика

Скрипт scripts/security_gate.py читает bandit.json, semgrep.json, pip-audit.json и trivy.json, нормализует находки, устраняет дубли (pip-audit выдаёт одну уязвимость под несколькими идентификаторами — GHSA, PYSEC, CVE) и применяет политику security-policy.json. Сборка блокируется, если есть хотя бы одна блокирующая находка или отсутствует отчёт одного из сканеров (сломанный сканер не должен означать «уязвимостей нет»). Сводка в Markdown публикуется в GITHUB_STEP_SUMMARY.

```
{
  "_comment": "Политика Security Gate: что блокирует слияние. Исключения — только с
обоснованием и сроком.",
  "block": {
    "bandit_min_severity": "MEDIUM",
    "bandit_min_confidence": "MEDIUM",
    "semgrep_severities": ["ERROR", "WARNING"],
    "sca_block_if_fix_available": true,
    "trivy_severities": ["CRITICAL", "HIGH", "MEDIUM"]
  },
  "ignore": [
    {
      "id": "B105",
      "path": "app/main.py",
      "reason": "Ложное срабатывание: строка 'bearer' — тип токена OAuth 2.0, не пароль",
      "expires": "2027-06-30"
    }
  ]
}
```

Листинг 9 — политика security-policy.json

Исключение допускается только адресно (идентификатор правила + файл), с обоснованием и сроком действия: по истечении срока находка снова блокирует сборку. Это защищает от бесконтрольного накопления «временных» подавлений.

7. Результаты сканирования исходной версии

Все проверки выполнены командой python scripts/run_local_scans.py before — она запускает те же инструменты с теми же параметрами, что и джобы пайплайна, и сохраняет отчёты в reports/before.

7.1. Тесты

Из пяти тестов три упали: пользователь с ролью USER получил доступ к экспорту аудита (200 вместо 403), токен, подписанный ключом злоумышленника, принят (200 вместо 401), а запрос с параметром zzz' OR 1=1 -- вернул всех пользователей.

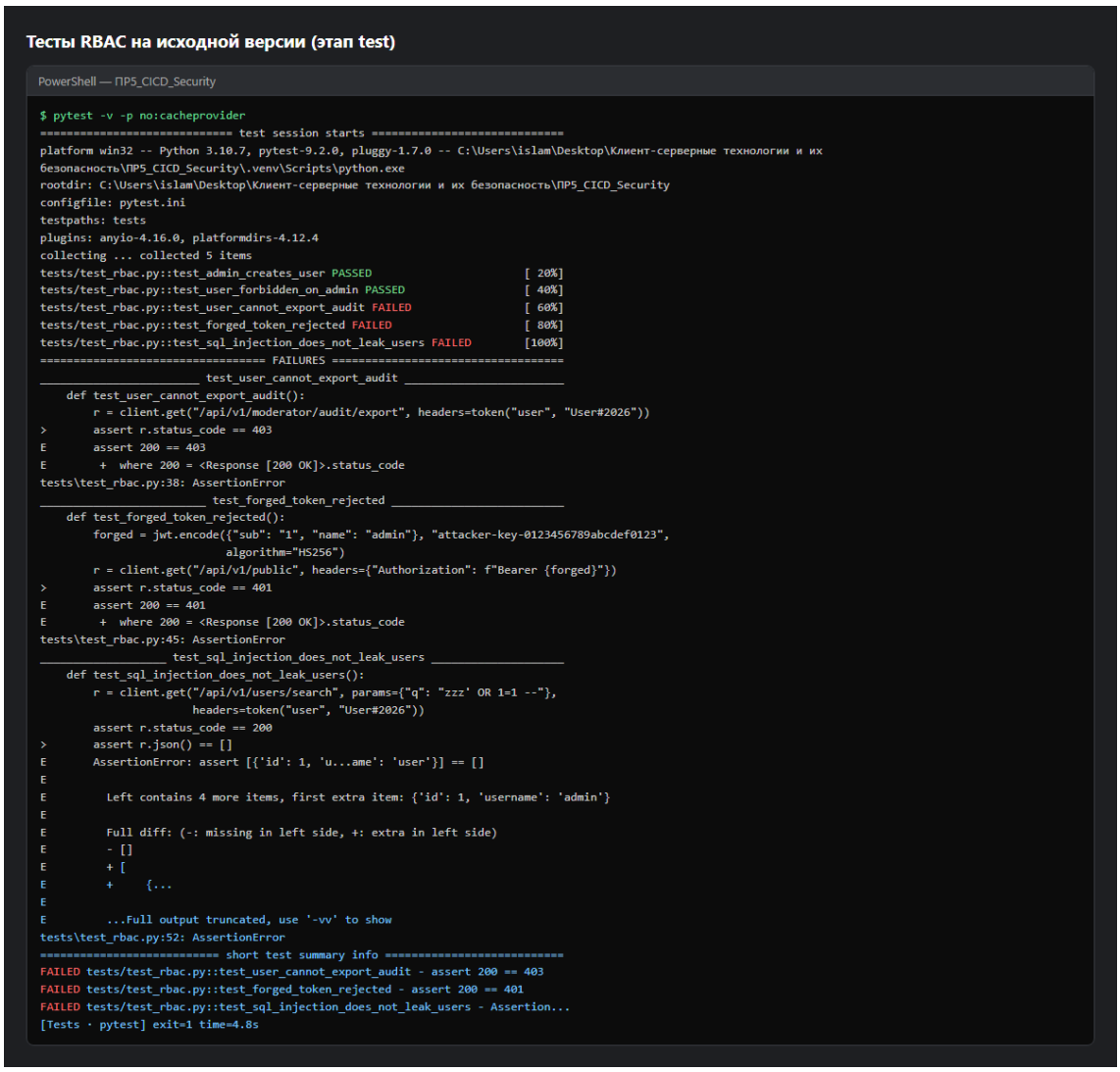


Рисунок 3 — pytest на исходной версии: 3 из 5 тестов не пройдены

7.2. SAST: Bandit

Bandit: исходная версия

PowerShell — ПР5_CICD_Security

```
$ bandit -r app -q
Run started:2026-11-02 14:28:28.557864+00:00
Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   Location: app\main.py:6:0
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'bearer'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   Location: app\main.py:57:70
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   Location: app\main.py:70:12
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   Location: app\main.py:82:10
>> Issue: [B506:yaml_load] Use of unsafe yaml load. Allows instantiation of arbitrary objects. Consider yaml.safe_load().
   Severity: Medium   Confidence: High
   CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html)
   Location: app\main.py:95:11
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interfaces.
   Severity: Medium   Confidence: Medium
   CWE: CWE-605 (https://cwe.mitre.org/data/definitions/605.html)
   Location: app\main.py:102:26
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'rbac-super-secret-key-2026'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   Location: app\security.py:10:13
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   Location: app\security.py:15:11
```

Рисунок 4 — Bandit: находки в исходной версии

Bandit нашёл 8 проблем, из них 5 блокирующих по политике (уровень и достоверность не ниже MEDIUM): SQL-инъекция (B608), shell=True (B602), yaml.load (B506), привязка к 0.0.0.0 (B104), MD5 (B324). Находки уровня LOW (импорт subprocess, строка «bearer») не блокируют сборку; «bearer» — ложное срабатывание, внесённое в исключения политики. Bandit отметил секрет в коде только как LOW и не обнаружил ни отключённую проверку подписи JWT, ни отсутствие проверки права на эндпоинте экспорта.

7.3. SAST: Semgrep

Semgrep: результаты сканирования (semgrep.json, before)

Файл:строка	Правило	Уровень	CWE
app/main.py:79	rbac-privileged-route-without-permission	ERROR	CWE-285: Improper Authorization
app/main.py:82	subprocess-shell-true	ERROR	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
app/main.py:95	avoid-pyyaml-load	ERROR	CWE-502: Deserialization of Untrusted Data
app/security.py:10	hardcoded-jwt-secret	ERROR	CWE-798: Use of Hard-coded Credentials
app/security.py:15	insecure-hash-algorithm-md5	WARNING	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
app/security.py:15	md5-used-as-password	WARNING	CWE-327: Use of a Broken or Risky Cryptographic Algorithm
app/security.py:26	jwt-python-hardcoded-secret	ERROR	CWE-522: Insufficiently Protected Credentials
app/security.py:31	jwt-decode-without-verification	ERROR	CWE-347: Improper Verification of Cryptographic Signature
app/security.py:31	unverified-jwt-decode	ERROR	CWE-287: Improper Authentication

Найдено: **9** · проверено файлов: 4 · наборы: p/python, p/owasp-top-ten, .semgrep/rbac-rules.yml

Рисунок 5 — Semgrep: находки в исходной версии

Semgrep нашёл 9 проблем — все блокирующие. Собственные правила обнаружили маршрут /api/v1/moderator/audit/export без require_permission (строка 79), декодирование JWT без проверки подписи и секрет в коде. Правила сообщества дополнительно указали на shell=True, yaml.load и MD5 для паролей. Bandit и Semgrep частично пересекаются, но каждый нашёл то, что пропустил другой, — поэтому в пайплайне оба.

7.4. SCA: pip-audit

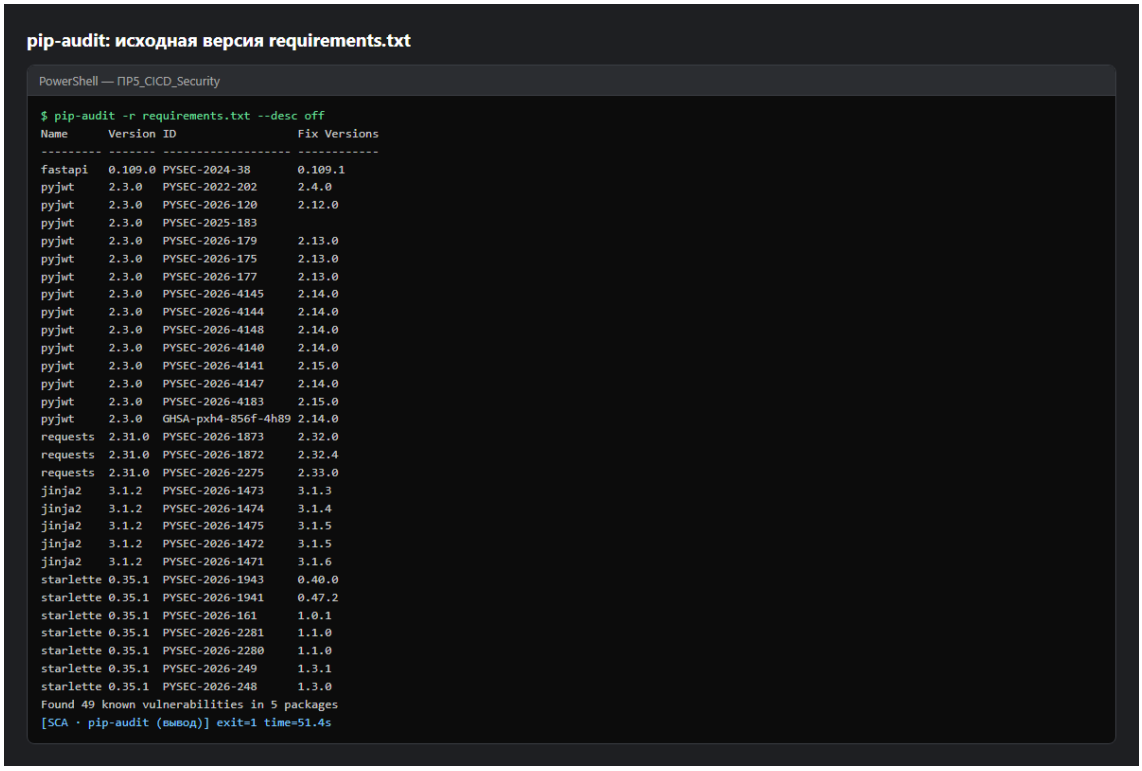


Рисунок 6 — pip-audit: уязвимые зависимости исходной версии

Таблица 8 — Уязвимости зависимостей исходной версии (после удаления дублей)

Пакет	Зависимость	Уязв.	Примеры идентификаторов	Исправлено в
fastapi 0.109.0	прямая	1	CVE-2024-24762	0.109.1
pyjwt 2.3.0	прямая	14	CVE-2022-29217, CVE-2026-32597, CVE-2025-45768 …	2.15.0
requests 2.31.0	прямая	3	CVE-2024-35195, CVE-2024-47081, CVE-2026-25645	2.33.0
jinja2 3.1.2	прямая	5	CVE-2024-22195, CVE-2024-34064, CVE-2024-56326 …	3.1.6
starlette 0.35.1	транзитивная (через fastapi)	7	CVE-2024-47874, CVE-2025-54121, CVE-2026-48710 …	1.3.1

pip-audit вывел 49 записей, после удаления дублей — 30 уникальных уязвимостей в 5 пакетах; 29 из них имеют исправленную версию и

блокируют сборку. Важно, что starlette в requirements.txt не указан: это транзитивная зависимость fastapi, и её уязвимости (в т.ч. CVE-2024-47874 — отказ в обслуживании при разборе multipart) видны только при анализе полного дерева.

7.5. Решение Security Gate

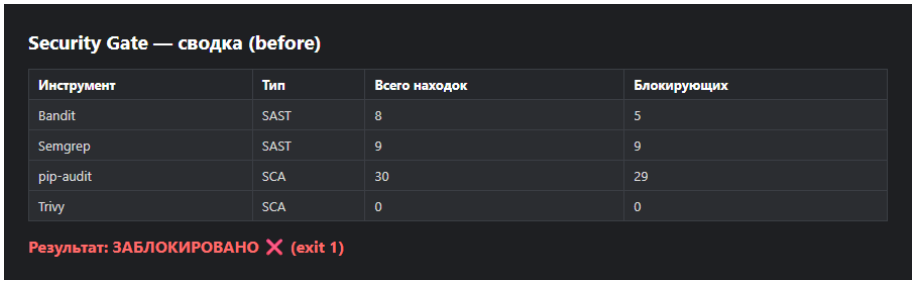


Рисунок 7 — Security Gate для исходной версии: сборка заблокирована

Итого блокирующих находок: 43. Gate завершился с кодом 1, что в CI означает красный статус и запрет слияния.

8. Устранение уязвимостей

Исправления внесены отдельным коммитом f14dc42. Соответствие находок и исправлений приведено в таблице.

Таблица 9 — Исправления по результатам сканирования

Находка	Инструмент	Исправление
SQL-инъекция (B608)	Bandit, тест	параметризованный запрос с плейсхолдером ?
shell=True (B602)	Bandit, Semgrep	CSV формируется модулем csv, оболочка не вызывается
Маршрут без проверки права	Semgrep (RBAC-правило), тест	Depends(require_permission("EXPORT_AUDIT"))
yaml.load (B506)	Bandit, Semgrep	yaml.safe_load
Секрет в коде	Semgrep	JWT_SECRET из окружения, проверка длины ≥ 32
JWT без проверки подписи	Semgrep, тест	jwt.decode(key, algorithms=["HS256"], require=[sub, exp])
MD5 для паролей (B324)	Bandit, Semgrep	PBKDF2-HMAC-SHA256, соль 16 байт, 600 000 итераций, compare_digest
Привязка к 0.0.0.0 (B104)	Bandit	HOST из окружения, по умолчанию 127.0.0.1
Уязвимые зависимости	pip-audit	обновление до актуальных версий, Dependabot

```
# Было
def decode_token(token: str) -> dict:
    # подпись не проверяется — можно подделать любой токен
    return jwt.decode(token, options={"verify_signature": False})


# Стало
def decode_token(token: str) -> dict:
    # подпись и срок действия проверяются, алгоритм — из белого списка
    return jwt.decode(token, SIGNING_KEY, algorithms=[ALGORITHM],
                      options={"require": ["sub", "exp"]})
```

Листинг 10 — проверка JWT до и после исправления

```
# Было
        # SQL-инъекция: параметр подставляется в строку запроса
        rows = conn.execute(
            f"SELECT id, username FROM users WHERE username LIKE '%{q}%'").fetchall()


# Стало
        # параметризованный запрос: значение передаётся отдельно от SQL
        rows = conn.execute("SELECT id, username FROM users WHERE username LIKE ?",
                            (f"%{q}%",)).fetchall()
```

Листинг 11 — поиск пользователей до и после исправления

```
# Было
@app.get("/api/v1/moderator/audit/export")
def audit_export(fmt: str = "csv", user=Depends(get_current_user)):
    # нет проверки разрешения EXPORT_AUDIT + инъекция команд ОС
    out = subprocess.check_output(f"echo audit export as {fmt}", shell=True)
    return {"result": out.decode().strip()}


# Стало
@app.get("/api/v1/moderator/audit/export")
def audit_export(user=Depends(require_permission("EXPORT_AUDIT"))):
    # экспорт формируется средствами Python, без вызова командной оболочки
    buf = io.StringIO()
    csv.writer(buf).writerows([["event"], ["login admin"], ["login moder"]])
    return {"result": buf.getvalue()}
```

Листинг 12 — эндпоинт экспорта аудита до и после исправления

Таблица 10 — Изменение версий в requirements.txt

Пакет	**Было**	**Стало**
fastapi	0.109.0	0.141.1
uvicorn	0.27.0	0.54.0
pydantic	2.5.3	2.14.2
PyJWT	2.3.0	2.15.1
PyYAML	6.0.1	6.0.3
requests	2.31.0	2.34.2
Jinja2	3.1.2	3.1.6

9. Повторное сканирование

Команда python scripts/run_local_scans.py after выполнена на исправленной версии, отчёты сохранены в reports/after.

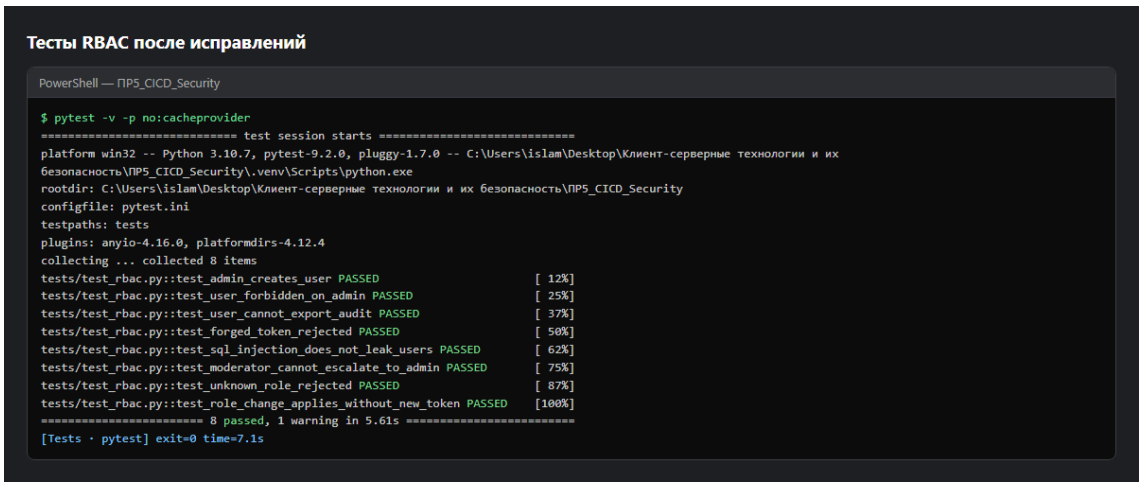


Рисунок 8 — pytest после исправлений: 8 из 8 тестов пройдены

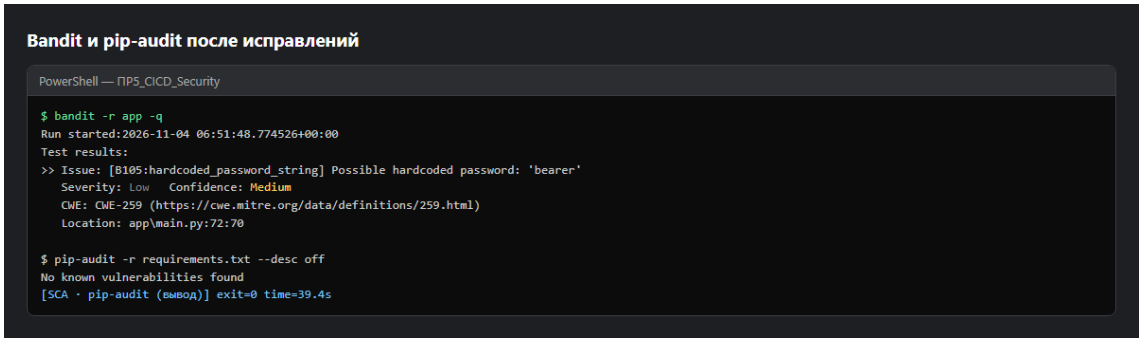


Рисунок 9 — Bandit и pip-audit после исправлений

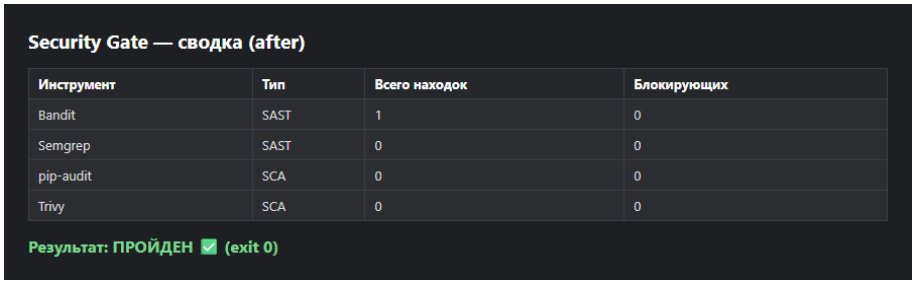


Рисунок 10 — Security Gate после исправлений: проверка пройдена

Bandit сообщает только о строке «bearer» уровня LOW — это тип токена OAuth 2.0, ложное срабатывание, внесённое в исключения политики с обоснованием и сроком. Semgrep и pip-audit находок не выдали.

Таблица 11 — Сравнение результатов до и после исправлений

Показатель	**До**	**После**
Тесты pytest (пройдено)	2 из 5	8 из 8 (+3 теста RBAC)
Bandit: всего / блокирующих	8 / 5	1 / 0
Semgrep: всего / блокирующих	9 / 9	0 / 0
pip-audit: уникальных CVE / блокирующих	30 / 29	0 / 0
Решение Security Gate	ЗАБЛОКИРОВАНО (exit 1)	ПРОЙДЕН (exit 0)

9.1. Запуск в GitHub Actions

Проект опубликован в публичном репозитории github.com/is1am/cicd-security-pipeline. Первый запуск пайплайна (run #1) завершился с ошибкой, и разбор его результатов оказался полезным сам по себе.

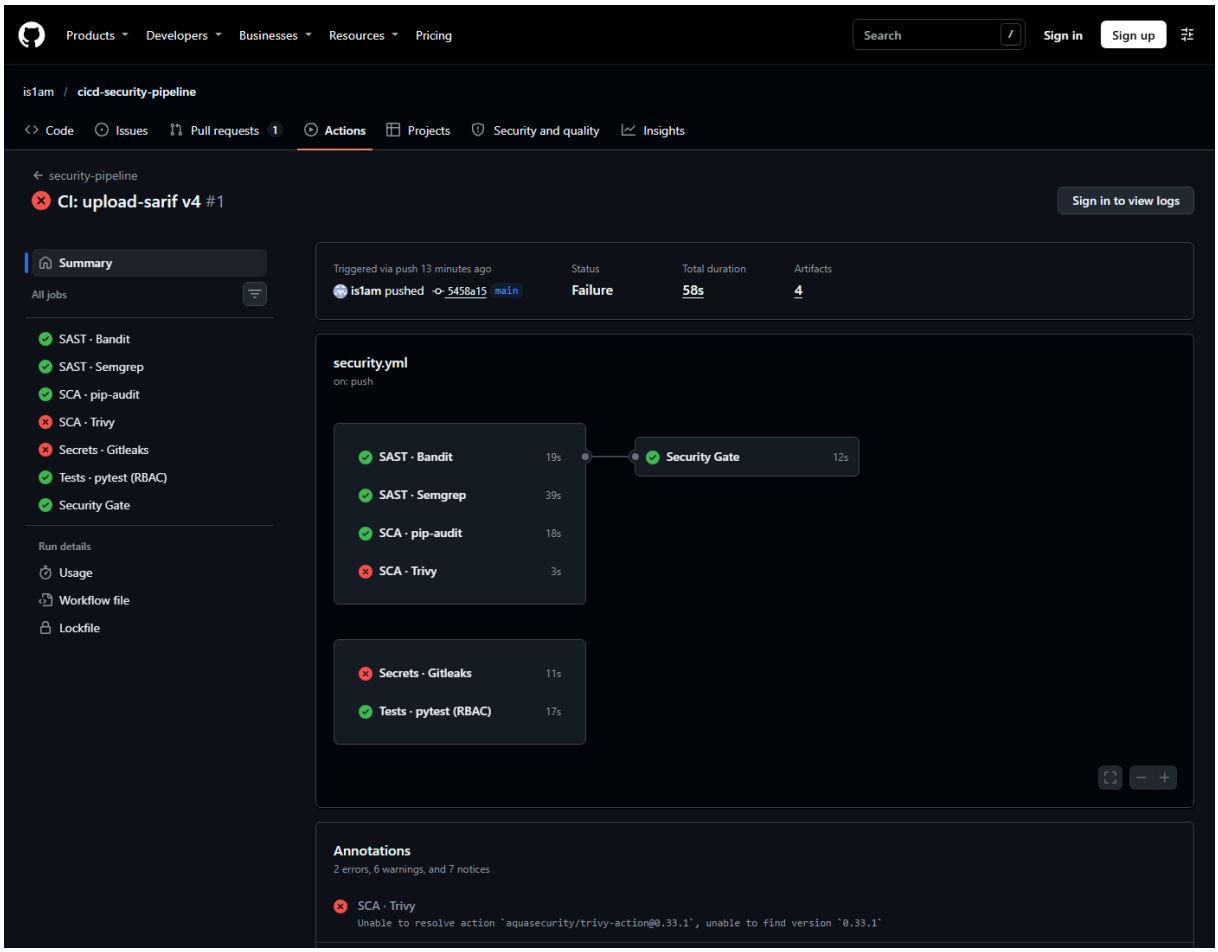


Рисунок 11 — Первый запуск в GitHub Actions (run #1): Trivy и Gitleaks не пройдены

Таблица 12 — Разбор первого запуска в GitHub Actions

Джоба	Результат	Причина и исправление
SCA · Trivy	ошибка	версия action указана как 0.33.1 вместо тега v0.33.1 — GitHub не нашёл action; исправлено на aquasecurity/trivy-action@v0.36.0
Secrets · Gitleaks	6 находок	приватный EC-ключ, JWT и generic-api-key в reports/before/pip-audit.json — это примеры из текстов advisory PyJWT, попавшие в закоммиченный отчёт сканера (ложное срабатывание); добавлен .gitleaks.toml с исключением для каталога reports/
Security Gate	пройден (ошибка Gate)	отчёт trivy.json отсутствовал, но Gate считал его необязательным; добавлен флаг --require-trivy, в CI отсутствие отчёта Trivy теперь блокирует сборку
Все джобы	предупреждения	actions на Node 20 устарели; обновлены до checkout@v7, setup-python@v7, upload-artifact@v7, download-artifact@v8, gitleaks-action@v3

Отдельно стоит отметить результат Gitleaks: строку rbac-super-secret-key-2026, которая действительно была секретом в исходной версии, он не обнаружил — её энтропия слишком низка для его эвристик. Этот секрет нашли только Semgrep (собственное правило hardcoded-jwt-secret и правило сообщества) и, с низким уровнем, Bandit. Это ещё один довод в пользу сочетания разных классов сканеров. Найденный в истории git секрет в реальном проекте нужно считать скомпрометированным и заменить, а не только удалить из текущей версии кода.

Второй запуск (run #8, после исправлений) прошёл полностью: все семь джоб завершились успешно, Security Gate разрешил сборку. Dependabot, включённый файлом .github/dependabot.yml, сразу после публикации открыл pull request «Bump fastapi from 0.141.1 to 0.162.0» — SCA работает и вне пайплайна.

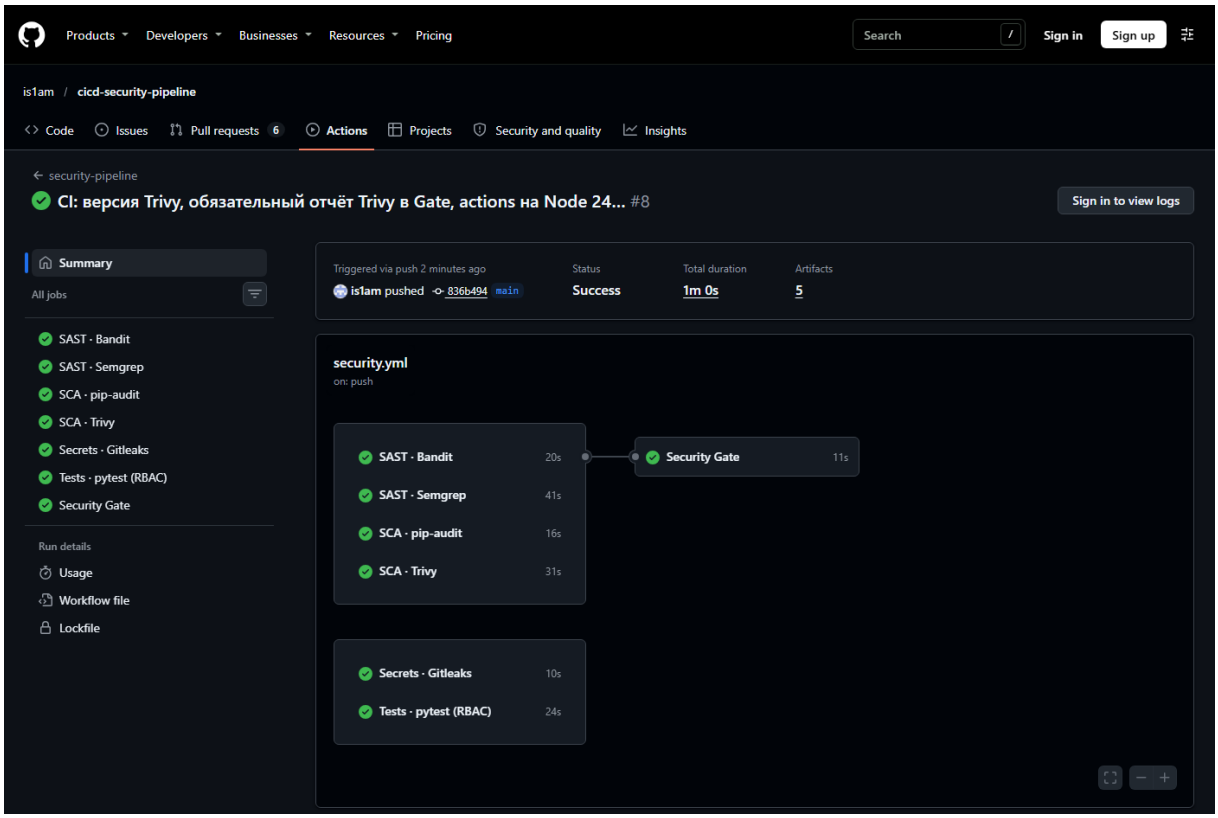


Рисунок 12 — Запуск в GitHub Actions после исправлений (run #8): все проверки пройдены

Следующий запуск (run #10, коммит с Postman-коллекцией и результатами API-тестов) снова остановил Gitleaks: 5 находок типа jwt — поддельный токен из негативного теста и токены тестового стенда в

reports/api/results.json. Локальная проверка Gitleaks 8.30.1 находок не давала. Причина — gitleaks-action по умолчанию устанавливает более старую версию Gitleaks, которая не поддерживает синтаксис [[allowlists]] и молча игнорировала исключения. Исправление: конфигурация переписана в совместимом формате [allowlist], а версия зафиксирована переменной GITLEAKS_VERSION: 8.30.1. Вывод: версии сканеров в CI и на рабочем месте разработчика должны совпадать, иначе одна и та же политика даёт разные результаты.

10. Тестирование ролевой модели в Postman (Negative Testing)

Тесты из шага 3 методических указаний оформлены как Postman-коллекция postman/RBAC_PR5.postman_collection.json и окружение RBAC_PR5.postman_environment.json (переменная baseUrl). Каждый запрос содержит тест pm.test(...) с ожидаемым статусом, а токены и идентификатор нового пользователя сохраняются в переменные окружения. Для воспроизводимого прогона используется скрипт scripts/run_collection.py: он запускает сервис на чистой БД, выполняет все запросы коллекции по порядку и проверяет статусы (аналог Newman). Ключевые сценарии (тесты 1–3) дополнительно выполнены вручную в Postman Desktop 12.29 против того же сервиса на http://127.0.0.1:8000 — их скриншоты приведены в подразделах 10.1–10.3.

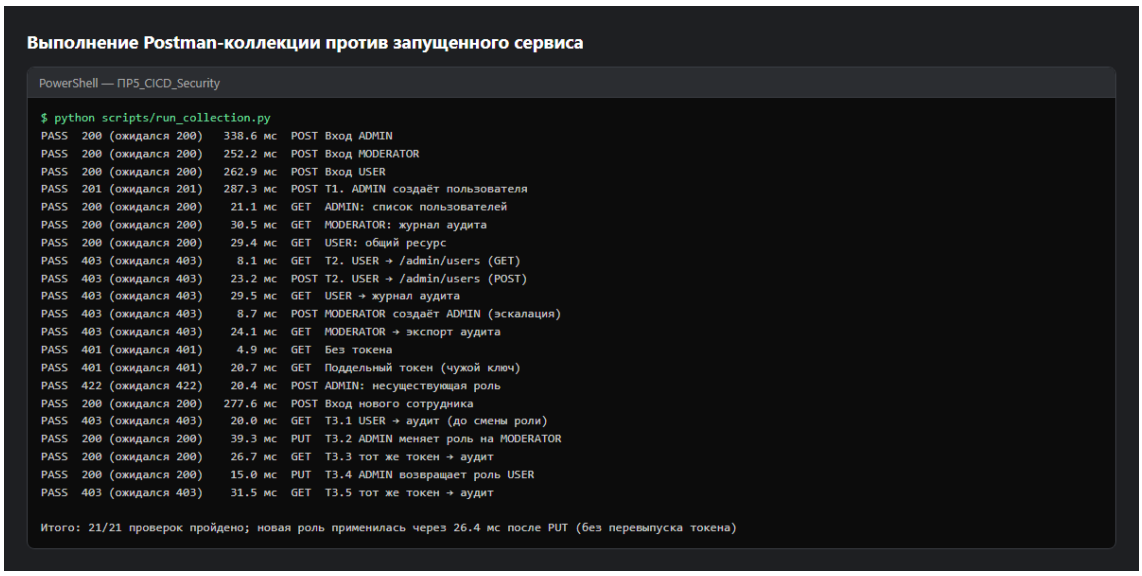


Рисунок 13 — Прогон Postman-коллекции: 21 из 21 проверок пройдено

Таблица 13 — Результаты запросов Postman-коллекции

Группа	Запрос	Ожидалось	Получено
Аутентификация	Вход ADMIN	200	200
Аутентификация	Вход MODERATOR	200	200
Аутентификация	Вход USER	200	200
Позитивные тесты	T1. ADMIN создаёт пользователя	201	201
Позитивные тесты	ADMIN: список пользователей	200	200
Позитивные тесты	MODERATOR: журнал аудита	200	200
Позитивные тесты	USER: общий ресурс	200	200
Негативные тесты	T2. USER → /admin/users (GET)	403	403
Негативные тесты	T2. USER → /admin/users (POST)	403	403
Негативные тесты	USER → журнал аудита	403	403
Негативные тесты	MODERATOR создаёт ADMIN (эскалация)	403	403
Негативные тесты	MODERATOR → экспорт аудита	403	403
Негативные тесты	Без токена	401	401
Негативные тесты	Поддельный токен (чужой ключ)	401	401
Негативные тесты	ADMIN: несуществующая роль	422	422
Смена роли «на лету»	Вход нового сотрудника	200	200
Смена роли «на лету»	T3.1 USER → аудит (до смены роли)	403	403
Смена роли «на лету»	T3.2 ADMIN меняет роль на MODERATOR	200	200
Смена роли «на лету»	T3.3 тот же токен → аудит	200	200
Смена роли «на лету»	T3.4 ADMIN возвращает роль USER	200	200
Смена роли «на лету»	T3.5 тот же токен → аудит	403	403

10.1. Тест 1 (позитивный)

Администратор входит в систему и создаёт пользователя. Сервер возвращает 201 Created и идентификатор новой записи.

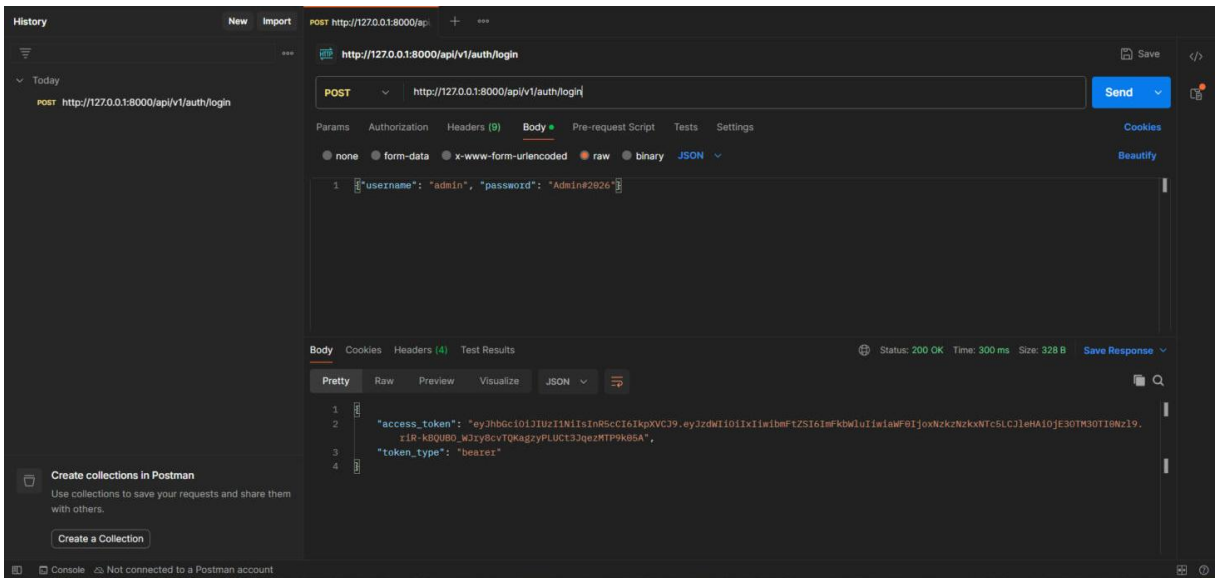


Рисунок 14 — Postman: вход администратора — 200 OK, выдан access_token

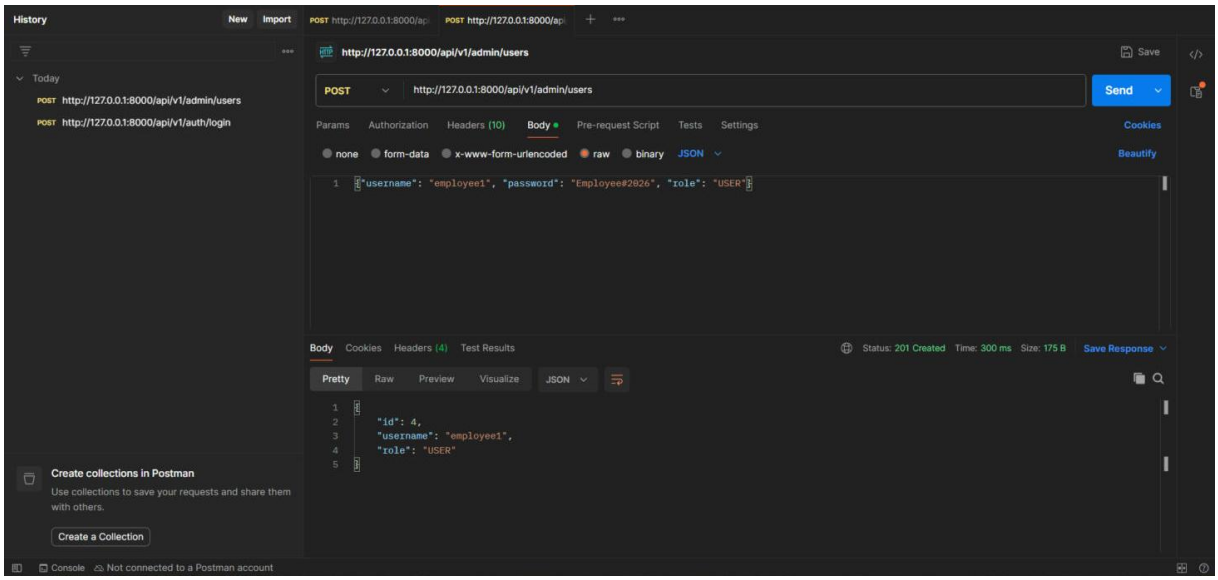


Рисунок 15 — Postman, тест 1: ADMIN создаёт пользователя — 201 Created

10.2. Тест 2 (негативный)

Рядовой сотрудник (USER) с валидным токеном обращается к /api/v1/admin/users. Аутентификация проходит, но у роли USER нет права MANAGE_USERS, и middleware возвращает 403 Forbidden и для чтения списка, и для попытки создать администратора.

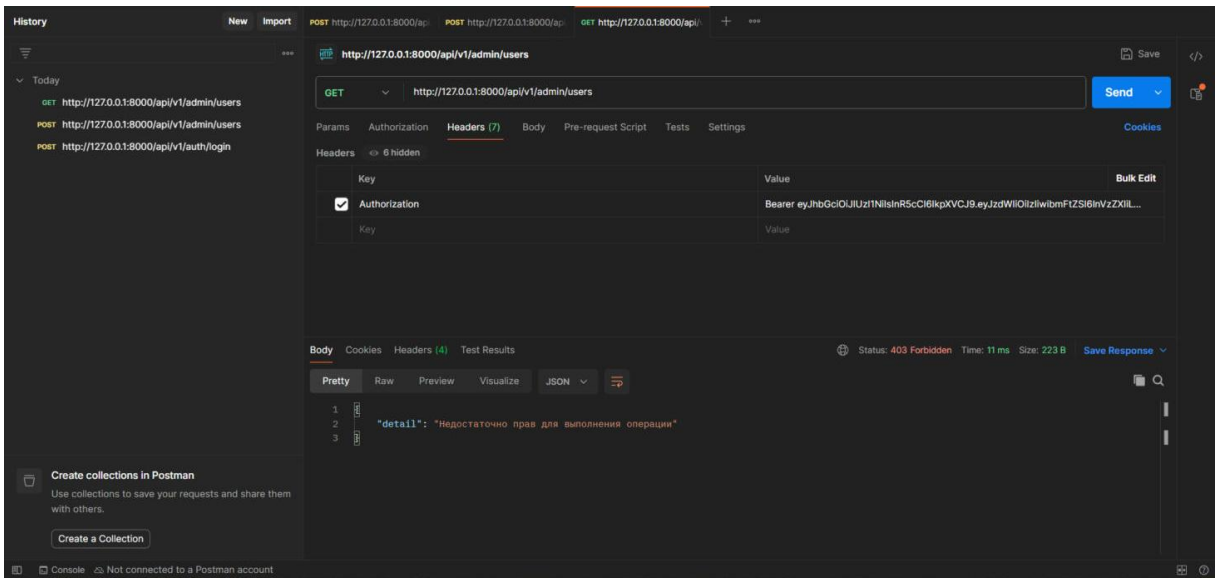


Рисунок 16 — Postman, тест 2: USER → GET /api/v1/admin/users — 403 Forbidden

Дополнительно проверены попытки эскалации привилегий: модератор пытается создать пользователя с ролью ADMIN и выгрузить аудит (право EXPORT_AUDIT есть только у ADMIN) — 403; запрос без токена и с токеном, подписанным чужим ключом, — 401; создание пользователя с несуществующей ролью SUPERUSER — 422.

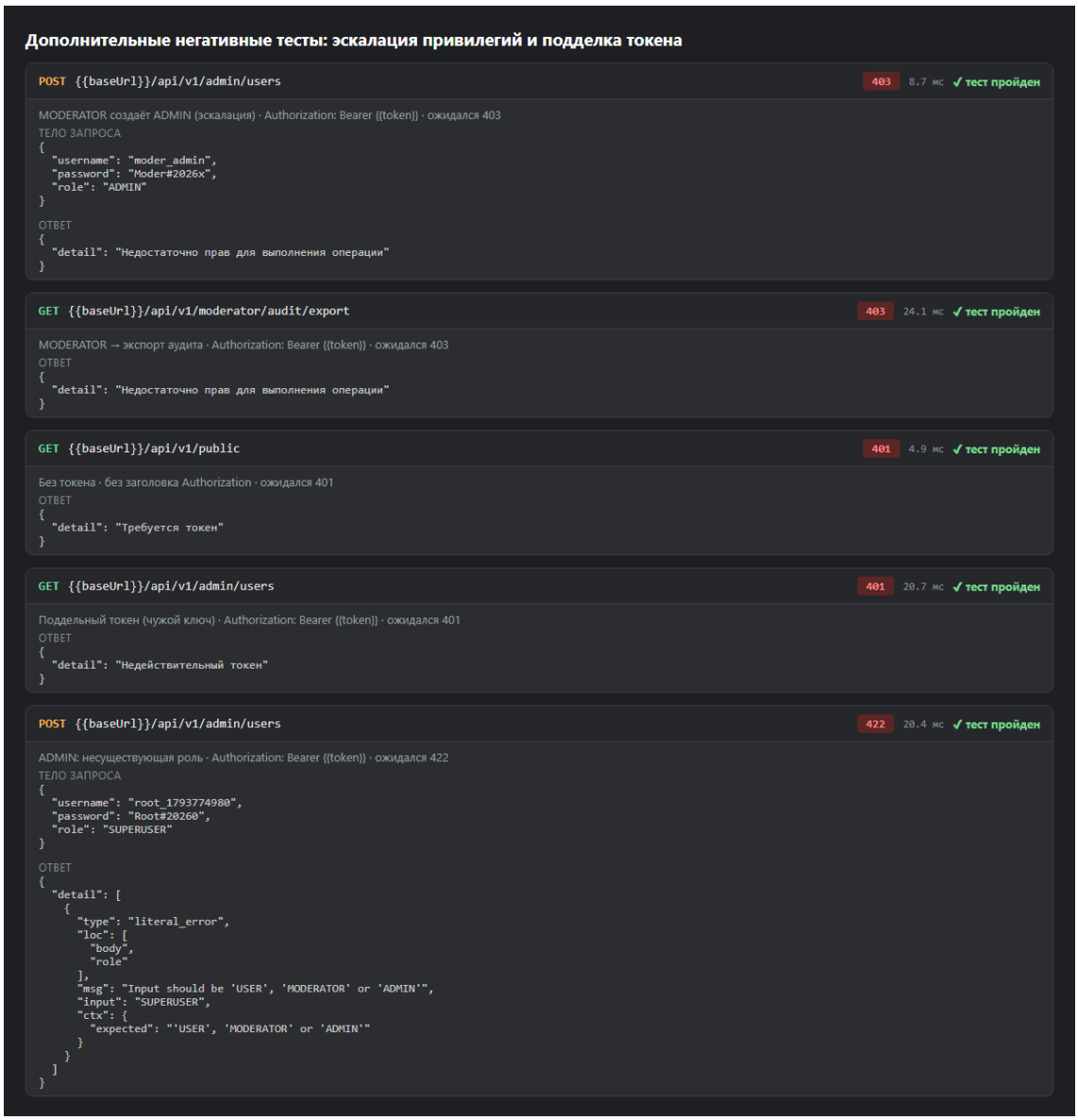


Рисунок 17 — Дополнительные негативные тесты: 403, 401, 422

10.3. Тест 3: смена роли «на лету»

Новый сотрудник получает токен с ролью USER и получает 403 на /api/v1/moderator/audit. Затем администратор меняет ему роль на MODERATOR (PUT /api/v1/admin/users/{id}/role, т.е. UPDATE таблицы user_roles), и сотрудник повторяет запрос с тем же токеном.

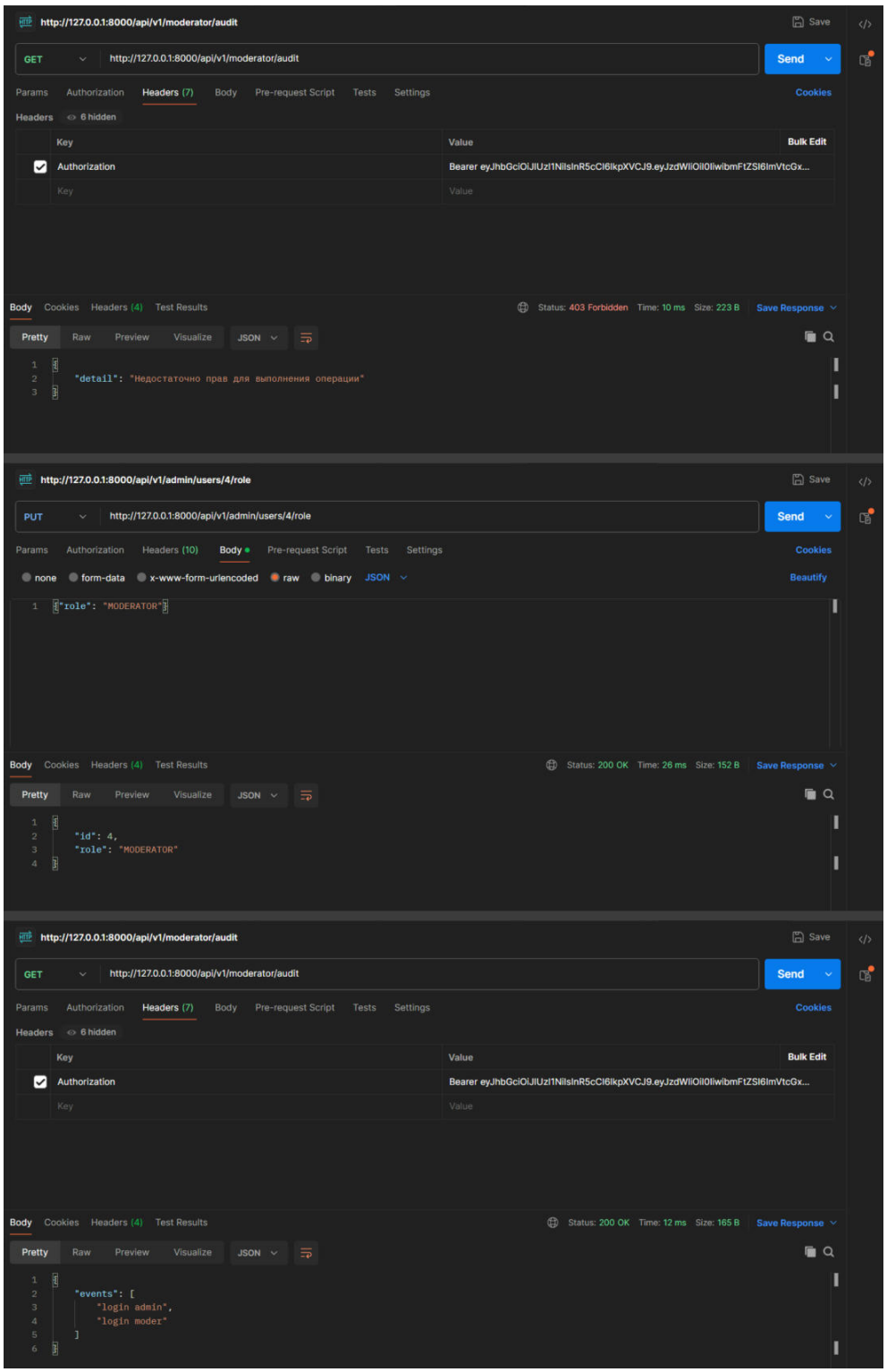


Рисунок 18 — Postman, тест 3: тот же токен — 403, смена роли на MODERATOR — 200, повтор — 200

Новая роль применилась на первом же запросе после изменения: в Postman повторный запрос с тем же токеном сразу вернул 200, а в прогоне коллекции интервал между ответом на PUT и успешным запросом составил 26.4 мс, то есть время одного HTTP-запроса; возврат роли USER (шаги T3.4–T3.5 коллекции) так же сразу вернул 403. Задержки нет, потому что токен

содержит только идентификатор пользователя (sub), а права читаются из БД при каждом запросе. Для сравнения ниже приведён альтернативный вариант, когда роль хранится в claims JWT.

Таблица 14 — Где хранить роли: сравнение подходов

Критерий	**Права из БД при каждом запросе (реализовано)**	**Роль в claims JWT**
Задержка применения новой роли	0 — следующий запрос	до истечения exp (15 мин) или до перевыпуска токена
Отзыв прав / блокировка	сразу (роль или is_active в БД)	токен действует до exp; нужен blacklist по jti
Нагрузка	1 SQL-запрос на каждый запрос API (кэш с коротким TTL)	проверка без обращения к БД
Риск	нагрузка на БД	окно, в котором уволенный или пониженный сотрудник сохраняет права

Для корпоративных систем, где своевременный отзыв прав важнее нескольких миллисекунд на запрос, выбран вариант с чтением прав из БД. Если роль всё же передаётся в токене, срок жизни access-токена должен быть коротким, а при смене роли — инвалидироваться refresh-токен и вестись список отозванных jti.

11. Сравнение GitHub Actions и GitLab CI

Таблица 15 — Реализация пайплайна в двух CI-системах

Критерий	**GitHub Actions**	**GitLab CI**
Описание пайплайна	.github/workflows/*.yml, jobs + needs	.gitlab-ci.yml, stages + needs
Готовые компоненты	Marketplace: trivy-action, gitleaks-action, upload-sarif	шаблоны include: SAST, Secret-Detection, Dependency-Scanning
Показ находок	SARIF → Security / Code scanning, аннотации в PR	artifacts:reports → виджет MR, Security Dashboard
Ограничения тарифа	Code scanning бесплатен для публичных репозиториев	Dependency Scanning и Security Dashboard — Ultimate; собственные джобы — на любом
Права токена	permissions на уровне workflow и job	CI_JOB_TOKEN, protected-переменные
Обновление зависимостей	Dependabot	Renovate / Dependency Scanning
Блокировка слияния	branch protection: required status checks	«Pipelines must succeed», approval policies

Пайплайн GitHub Actions проверен реальными запусками в репозитории github.com/is1am/cicd-security-pipeline. Конфигурация GitLab CI

реализует ту же логику на тех же инструментах и командах, но в GitLab в рамках работы не запускалась.

12. Выводы

1. Построен конвейер безопасности для GitHub Actions и GitLab CI, который при каждом изменении и еженедельно по расписанию выполняет SAST (Bandit, Semgrep), SCA (pip-audit, Trivy), поиск секретов (Gitleaks / Secret Detection) и тесты. Решение о допуске изменения принимает единый Security Gate по явной политике.

2. На исходной версии RBAC-сервиса пайплайн обнаружил 43 блокирующих находок: инъекции SQL и команд ОС, небезопасную десериализацию, секрет в коде, отключённую проверку подписи JWT, MD5 для паролей и 30 известных уязвимостей в 5 пакетах, включая транзитивную зависимость starlette. После исправлений все проверки пройдены.

3. Ни один инструмент не нашёл всё в одиночку: Bandit не заметил отключённую проверку подписи JWT и отсутствие проверки права; правила сообщества Semgrep не знают о require_permission. Комбинация нескольких сканеров и собственных правил, учитывающих архитектуру проекта, существенно повышает полноту анализа.

4. Ролевая модель спроектирована по NIST RBAC (Core и иерархия через состав прав): права назначаются ролям, роли — пользователям, проверка выполняется на сервере по праву, а не по имени роли, и по умолчанию запрещено всё, что не разрешено явно. Postman-тесты (21 из 21 пройдено) подтвердили: USER получает 403 на административных ресурсах, модератор не может создать администратора и выгрузить аудит, поддельный токен отклоняется с 401, а неизвестная роль — с 422.

5. Правильно спроектированный RBAC предотвращает эскалацию привилегий, потому что у пользователя нет пути получить право, не выданное его роли: назначать роли может только обладатель MANAGE_USERS, роль в запросе ограничена белым списком, а права

читаются из БД при каждом запросе. Поэтому понижение роли или блокировка учётной записи действуют сразу (тест 3 — без задержки), и у скомпрометированной или уволенной учётной записи не остаётся окна, в котором старые права ещё работают.

6. Для RBAC-систем особенно важно собственное правило SAST, проверяющее, что каждый привилегированный маршрут защищён проверкой права. Оно превращает ошибку контроля доступа (эскалацию привилегий) из находки пентеста в автоматическую проверку каждого pull request. Негативные тесты (403 для USER, 401 для поддельного токена) дополняют SAST, подтверждая поведение на работающем приложении.

7. SCA должен анализировать полное дерево зависимостей и запускаться по расписанию: уязвимости появляются в библиотеках без изменения кода проекта. Автоматические PR Dependabot сокращают время от публикации CVE до обновления.

8. Эффективность пайплайна определяют не только сканеры, но и процесс: обязательный статус в branch protection, минимальные права токена CI, зафиксированные версии инструментов, блокировка при отсутствии отчёта и исключения только с обоснованием и сроком действия.